

COMPUTER ARCHITECTURE



Hamid Sarbazi-Azad

Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

Virtual Memory

In most modern processors the physical main memory is not as large as the space spanned by the address generated by the processor.

When a program does not completely fit into the main memory, the parts not currently being executed are stored on secondary storage (disk). Of course, all parts of a program that are eventually executed are first brought into the main memory. When a new segment is moved to a full main memory, it must replace another segment. Such movements of data and code is done automatically by OS.

Techniques to automatically move needed data and program blocks into the physical main memory are called **Virtual Memory** techniques.

Virtual Memory

Programs, and hence processor, reference an instruction and data space that is independent of available physical main memory space. The binary address generated by processor is called **virtual address** or **logical address**.

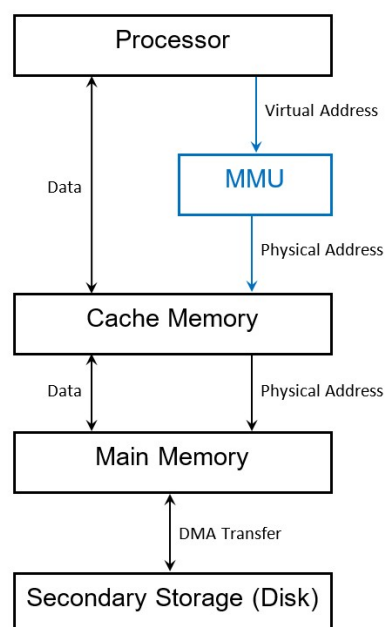
Virtual address is translated to physical address by a combination of hardware and software components. If a virtual address refers to part of program/data currently in the physical main memory, then the content of appropriate location is accessed immediately. Otherwise, it is taken to the main memory before it can be used.

Virtual Memory

A specific hardware unit called Memory Management Unit (MMU), translates virtual addresses into physical address.

When desired data/program code are in main memory, they are fetched and used. If not, MMU causes OS to bring data into the main memory from disk.

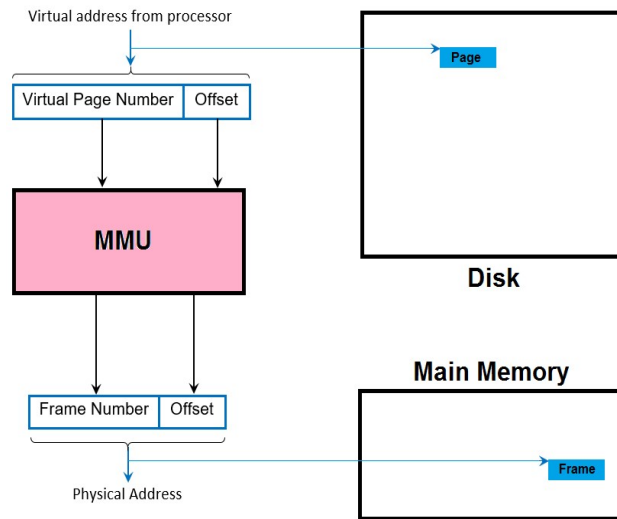
Data movement between disk and main memory is usually done through DMA.



Virtual Memory

Virtual address space (including disk space) and main memory address space are divided into fixed-size (a power of 2) segments.

These fixed-size segments are called *page* at secondary storage side and *frame* at main memory side.



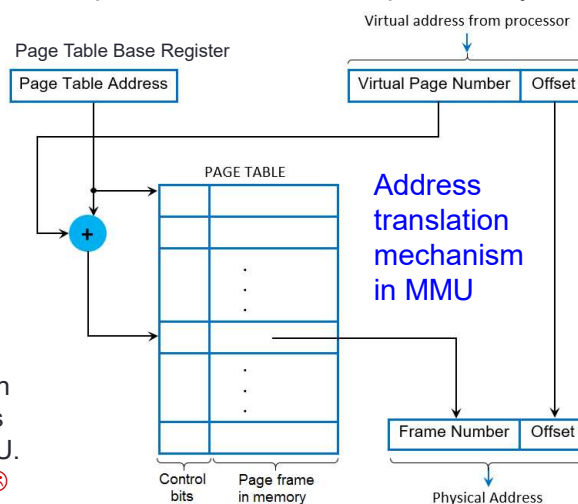
Virtual Memory

Virtual address (of data or program) has two parts: Virtual Page Number (MSBs) and Offset (LSBs). Offset specifies the location of a particular byte (or word) within a page.

Information about main memory location of each page is kept in Page Table In main memory.

Control bits (Presence bit, Modified bit, History bits, Access Rights bits, ...) are used by OS to manage users needs.

Page Table is used for each memory read/write. So, it is better to have it inside MMU. But it is big → impossible 😞



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8+: Virtual Memory Organization

7

Virtual Memory

One way is to keep a portion of it in a small buffer inside MMU called Translation Lookaside Buffer (TLB). It works like a cache for Page Table and can be arranged as Fully-Associative/Set-Associative/Direct-Mapped structure. For each virtual address, first, TLB is checked. If found the address is translated and the corresponding frame in the main memory is accessed. Otherwise, the needed entrees are taken from Page Table to TLB.

Like in caches, replacing valid entries in TLB can be done with a proper scheme, e.g. FIFO, LRU, ...

The diagram illustrates the TLB operation. A 'Virtual address from processor' is split into a 'Virtual Page Number' and an 'Offset'. The 'Virtual Page Number' is used to check the 'TLB' (Translation Lookaside Buffer), which is a table with columns for 'Virtual page number', 'Control bits', and 'Page frame in memory'. If the entry is a 'Hit', the 'Page frame in memory' is used to find the 'Frame Number' in the 'Page Table' (which also has an 'Offset' column). The 'Frame Number' and 'Offset' are then combined to form the 'Physical Address'. If the entry is a 'Miss', the 'Virtual page number' is used to look up the 'Page frame in memory' in the 'Page Table'.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8+: Virtual Memory Organization

8

Virtual Memory

When OS alters entries in Page Table, it must also update the copies in TLB. One easy way is to invalidate the corresponding entries in TLB.

When a program generates an access request to a page that is not in main memory, a **page fault** occurs.

MMU asks OS to intervene by raising an exception (internal interrupt). The control is given to OS and it copies the needed page from disk into the main memory and returns control to the interrupted task.

Since it may take a long time, OS can suspend the execution of the task caused the page fault and begin the execution of another task whose pages are in the main memory. When a new page is taken into main memory that is already full, OS should decide to replace one of the resident pages in the main memory. Again, there are different policies for it: FIFO, LRU, ...

This diagram is identical to the one on slide 7, showing the flow from a virtual address to a physical address through the TLB and page table.

Virtual Memory

Memory Management Requirements:

Virtual memory management routines are part of the OS. They reside in **system space** of virtual memory space that is separate from the virtual space where user applications (program+data) reside, called **user space**.

There might be several user spaces each for one user. This is arranged by providing a separate page table for each user. OS can switch from one user space to another by changing the content of Page Table Base Register (part of context switching).

The main memory is shared by active pages of system space and several user spaces, but only the pages belonged to one of these spaces are accessible at any given time.

Virtual Memory

Memory Management Requirements (cont.):

Each page must be protected from the accesses by other spaces. To this end, processors can work in **Supervisor** or **User mode**. OS routines are executed in Supervisor mode while normal programs of users are executed in User mode.

In User mode, some machine instructions (called **privileged instructions**, designated to manage virtual memory) cannot be executed. So, a user cannot modify the Page Table or other important information of other users or OS.

It is sometimes desirable for one user to have access to certain pages belonging to another user. OS can arrange for these pages to appear in both spaces. Such shared pages will have entries in both page tables of the users. For example, one user might give read/write access to a shared page while other only has read access.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8+: Virtual Memory Organization

11

Virtual Memory

Virtualization can be further expanded to two levels: Segment and Page, where each user has one segment of some pages.

The diagram illustrates the address translation process in a Segmented-Page system. A virtual address from the processor is divided into three parts: Segment, Page, and Offset. The Segment part is used to locate a segment in the SEGMENT TABLE. The Segment TABLE entry provides a base address, which is added to the Page part to find the corresponding page in the PAGE TABLE. The PAGE TABLE entry provides a base address for the page frame in memory. Finally, the Offset part is added to this base address to produce the Physical Address.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8+: Virtual Memory Organization

12

Virtual Memory

In a Segmented-Page virtual memory, to speedup address translation process, the segment part of virtual address may indicate a **segment register** that points to a specific Page Table.

The diagram illustrates the address translation process in a Segmented-Page system using segment registers. A virtual address from the processor is divided into three parts: SegReg, Page, and Offset. The SegReg part points to a specific register in the Register File, which contains pointers to various Page Tables. The Page part is then used to find the corresponding page in the selected Page Table. The PAGE TABLE entry provides a base address for the page frame in memory. Finally, the Offset part is added to this base address to produce the Physical Address.

(c) Hamid Sarbazi-Azad

Computer Structure & Language -- Lecture#5: Addressing modes

13

Multiuser support of segmented addressing

In a multiuser machine where users come and go anytime they want, OS has to create an environment that all users thinks they can easily work with computer and run their program.

- Each user take a portion of memory when login.
- The size of that portion depends on user's category/level.
- On logout that portion of memory is freed.
- Among active users, CPU is time multiplexed to execute their code by the OS.

```
graph TD; U1((User #1)) --> OS([Operating System]); U2((User #2)) --> OS; Dots[...] --> OS; UN((User #N)) --> OS; OS --> CS[Computer System];
```

(c) Hamid Sarbazi-Azad

Computer Structure & Language -- Lecture#5: Addressing modes

14

Multiuser support of segmented addressing

- The OS keeps a table of active users' information.
- When a user is taking control of CPU, the vital data (registers' contents including segment or page register, PC, Status Register, ...) called **Context** of the previous user is stored into the memory and the Context of the next user is loaded from memory.
This is called **Context Switching**.
- Context switching is time consuming
→ longer time slice can better utilize CPU.
- Some modern designs use large Register files to implement zero-latency context switching (GPUs are examples).

User ID	Segment Start Address	Segment End Address	Access Rights /Priorities	Pointer to User's Context	...
User #1	0107FD5D	01FFFFFF	Write/Read/Supervi	00010200	...
User #2	000F0000	001FFFFFF	Write/Read	00010300	...
...	...	...	...	...	...
User #N	F00FDDC0	FF000000	Write/Read/Supervi	00010900	...

END OF SLIDES

QUESTIONS?